

Summary

find	search for files or directories in a directory
head	prints the first few lines in the file
tail	prints the last few lines in the file
wc	print newline, word, and byte counts for each given file
cut	Print selected parts of lines from each FILE
grep	print lines that match patterns

Summary

	pipe
">", ">>"	output redirection
"<"	input redirection
wget	download file from <link>
chmod	Change File Mode
tar	compress and decompress

Homeworks

1. Create a directory named "hw2".
2. Move to "hw2" directory, and download this file (SRR25653406.fasta.tar.gz) using wget command:
https://github.com/UeenHuynh/MGMA_2024/raw/main/lecture_2/2.%20basic%20Linux%20commands%20part%202/SRR25653406.fasta.tar.gz
3. Decompressing this file SRR25653406.fasta.tar.gz using tar command.

Additional information about FASTA format (The file has the extension .fasta, .faa, .fna, etc.):

FASTA format is a text-based format, contains two lines:

- First line: is the comment (description) line.
 - + **Always starts with ">" sign (This information is useful for the exercise).**
 - + Give basic information about the sequence (nucleotide or amino acid).

- Second >M35309.1 E.coli 16S rRNA fragment string.

Example: GGCATGAAGACACACTGCTAACTCCGAATACGCACAAGCCCGTAATGGAGCGACGGTGGGCCTTGTTCCC
GTGCCCCGATGTGGGGTGGAGGTGACTGTGGGTTGTGATATTCGGGGAGGCAAAAGAAGTAGCGAGTCTA
ACCTTGCTTACCACTTTGCCTAATACGGGAAACG

(<https://www.ncbi.nlm.nih.gov/nuccore/M35309.1?report=fasta>)

=> Let's call this single-sequence FASTA format (https://en.wikipedia.org/wiki/FASTA_format), so if a text file contains a single-sequence FASTA format, this file is called a single-sequence FASTA file.

If a text file contains two or more single-sequence FASTA format, this file is called a multiple-sequence FASTA file or multi-FASTA file.

Homeworks

An example of multi-FASTA format :



(https://www.researchgate.net/figure/A-sample-of-the-Multi-FASTA-file_fig1_309134977)

Additional information for this file: SRR25653406.fasta (after decompressing using tar command).

- This is a multi-FASTA file. It is converted from fastq to fasta format file (Don't worry about fastq format in this exercise), the original fastq format file is obtained from the sequencing machine.
- Each single-sequence FASTA in this file is information about **a read** - a piece of DNA sequence obtained from the sequencing machine:
 - + First line: some information about the sequencing platform of this file, that's all, don't worry about the details.
 - + Second line: DNA sequence of that **read**, a character string consisting of 4 types of letters: A, T, G, C representing 4 types of nucleotides. In addition, there is the letter N, which means 1 of 4 possible types of nucleotides, because the sequencing machine cannot identify which type of nucleotide it is.

Homeworks

A single-sequence FASTA in this file:

```
>SRR25653406.lite.3.8 M02606:41:0000000000-J3RLF:1:1101:8131:2344 length=200  
TTTGGTCCAGCAGGCTATTGCTGGGAATTGTTAGCTTTTCGGGATCCATTTTATTGGGCGGTGTGCTCATCTCTACCATTTC AACATTATAGAGCGGCGTGTGG  
ATGTGGTAAACACGGGGCGAATGACTTACCGCAACATAACTCAACACTATGTATTGATAGGCTTCAATGA ACTAACTATCAATATGATACGCGAA
```

(The DNA sequence has 2 lines because the screen is not enough to display 1 line, so it is split into 2 lines)

4. How many lines does this file have? Using: wc.
5. How many reads does this file have? Using: grep for the search “^>” means the line starts with the character ">", and -c option.
6. How many reads **do not have any N**? Using: grep with -v, -c option; pipe.
7. Create a file named "id_read.txt" containing only the first line (the line contains information) of all reads. Using: grep with “^v”; output redirection.

Homeworks

8. Create a file named "part_of_id_read.txt" containing **a part of the first line** (the line contains information) of **all reads**. Using: grep; pipe; cut; output redirection.

Two lines of 1 read:

```
>SRR25653406.lite.3.8 M02606:41:000000000-J3RLF:1:1101:8131:2344 length=200  
TTTGGTCCAGCAGGCTATTGCTGGGAATTGTTAGCTTTTCGGGATCCATTTTATTGGGCGGTGTGCTCATCTCTACCATTTCCAACATTATAGAGCGGCGTGTGG  
ATGTGGTAAACACGGGGCGAATGACTTACCGCAACATAACTCAACACTATGTATTGATAGGCTTCAATGAACTAACTATCAATATGATACGCGAA
```

(The line containing the DNA sequence has 2 lines because the screen is not enough to display 1 line, so it is split into 2 lines).

a part of the first line of 1 reads:

```
M02606:41:000000000-J3RLF:1:1101:8131:2344
```



Cancer_Epigenetic 2025

Introduction to bash script (1)

Adapted and Updated from MGMA 2024

Huyha

Nov 16 2025

Email: hagiahuy311@gmail.com

Contents

1. What is bash script?
 - +Write your first bash script (Hello world)
 - +Datastream
2. Function
3. Variables, argument
4. Basic math /Question 1
5. IF Statement (Condition) + Case statement Condition/ Classwork
6. Exit Code
7. Cheat sheet/Use AI to learn code/Home work
8. Loop
9. Classwork
10. Awk and application
11. Homework

More: [Extension Linux bash script learning](#)

MGMA_2024

Contents

1. More grep
2. More combining and redirection
3. What is bash script?
+Write your first bash script (Hello world)
+Datastream
4. Function
5. Variables, argument
6. Basic math /Question 1
7. IF Statement (Condition) + Case statement Condition/ Classwork
8. Exit Code
9. Loop
10. Practice day
11. Basic Sed + Basic Awk 1
12. Awk 2

More grep

```
echo -e  
">seq1|human|chr1\n\nATTGGCCTAG\n\n>seq2|mouse|chr2\n\nATTGGATTAC\n\n>seq3|fly|chrM\n\nGCTAGGCCTA\n\n>seq4|human|chr1\n\nTTAGGCCTAG" > samples.fasta
```

- **Start of line:** Find all FASTA headers.

```
grep "^>" samples.fasta
```

- **End of line:** Find sequences ending in "TAG"

```
grep "TAG$" samples.fasta
```

 - OR (use grep -E):

- **Find "human" OR "mouse".**

```
grep -E "human|mouse" samples.fasta
```

- **Character set:** Find sequences with a "C" or "G".

```
grep -E "[ATGC]+[CG]+[ATGC]+$" samples.fasta
```

- **Invert match:** Get everything except headers.

```
grep -v "^>" samples.fasta
```

Step 1: Create `genes.gff`

Bash

```
cat << 'EOF' > genes.gff
scaffold_1  Prodigal  gene  100  500  .  +  .  ID=gene001
scaffold_1  Prodigal  CDS   100  500  .  +  0  ID=cds001;Parent=gene001
scaffold_1  Prodigal  gene  650  900  .  -  .  ID=gene002
scaffold_1  Prodigal  CDS   650  900  .  -  0  ID=cds002;Parent=gene002
scaffold_2  AUGUSTUS  gene  200  800  .  +  .  ID=gene003
scaffold_2  AUGUSTUS  mRNA  200  800  .  +  .  ID=mrna003;Parent=gene003
scaffold_2  AUGUSTUS  CDS   200  350  .  +  0  ID=cds003.1;Parent=mrna003
scaffold_2  AUGUSTUS  CDS   450  800  .  +  0  ID=cds003.2;Parent=mrna003
scaffold_2  Prodigal  gene  1000  1200  .  +  .  ID=gene004;Note=hypothetical protein
scaffold_2  Prodigal  CDS  1000  1200  .  +  0  ID=cds004;Parent=gene004
EOF
```

Step 2: Create `sequences.fasta`

Bash

```
cat << 'EOF' > sequences.fasta
>seq1_gene001 organism=Kitasatospora
ATGCGTGACTAGCTAGCTAGCTAG
CTAGCTAGCTAGCTAGCTAGCTAG
>seq2_gene002 organism=Levilactobacillus
TTACGACTAGCTAGCTAGCTAGCT
GCTAGCTAGCTAGCTAGCTAGCTA
>seq3_gene003
ATGGGGCTAGCTAGCTAGCTAGCT
EOF
```

The Tasks

1. Find all the "gene" features in the `genes.gff` file.
2. Count how many "CDS" features are in `genes.gff`.
3. Find all features on "scaffold_1" in `genes.gff` use cut to extract features
4. Show all lines in `genes.gff` that *do not* contain "Prodigal".
5. Find the header for "seq2_gene002" in `sequences.fasta` and show its line number.
6. Find all headers in `sequences.fasta` that mention an "orgAnism" (case-insensitive).
7. **Challenge 1:** In `genes.gff`, find all lines that are *either* "gene" or "mRNA". (Hint: Use the `-E` flag and the `|` symbol, which means "OR").
8. **Challenge 2 (Pipe):** Count the total number of sequences in `sequences.fasta`. (Hint: First, `grep` for the header lines, then "pipe" that output to another command: `wc -l`. The `wc -l` command counts lines).

case-sensitive

abc $\neq$ ABC

case-insensitive

abc = ABC

Core Concepts for Combining

We'll use the pipe (|) to chain commands, and introduce four new, essential tools.

- **| (Pipe)**: Takes the output of the command on its left and "pipes" it as the input to the command on its right.
- **> (Redirect)**: Takes the output of the command on its left and **overwrites** a file with it. (Be careful! This deletes what's in the file.)
- **>> (Append)**: Takes the output of the command on its left and **adds** it to the end of a file. This is safer for adding data.
- **sort**: Sorts lines of text alphabetically.
- **uniq**: Removes duplicate *adjacent* lines. (This is why you almost always use **sort** first).
- **wc -l**: **W**ord **C**ount with the **-l** (lines) flag. It counts the number of lines it receives.
- **cut**: Extracts specific columns from text.

Step 1: Create `genes.gff`

Bash

```
cat << 'EOF' > genes.gff
scaffold_1 Prodigal gene 100 500 . + . ID=gene001
scaffold_1 Prodigal CDS 100 500 . + 0 ID=cds001;Parent=gene001
scaffold_1 Prodigal gene 650 900 . - . ID=gene002
scaffold_1 Prodigal CDS 650 900 . - 0 ID=cds002;Parent=gene002
scaffold_2 AUGUSTUS gene 200 800 . + . ID=gene003
scaffold_2 AUGUSTUS mRNA 200 800 . + . ID=mrna003;Parent=gene003
scaffold_2 AUGUSTUS CDS 200 350 . + 0 ID=cds003.1;Parent=mrna003
scaffold_2 AUGUSTUS CDS 450 800 . + 0 ID=cds003.2;Parent=mrna003
scaffold_2 Prodigal gene 1000 1200 . + . ID=gene004;Note=hypothetical protein
scaffold_2 Prodigal CDS 1000 1200 . + 0 ID=cds004;Parent=gene004
EOF
```

Step 2: Create `sequences.fasta`

Bash

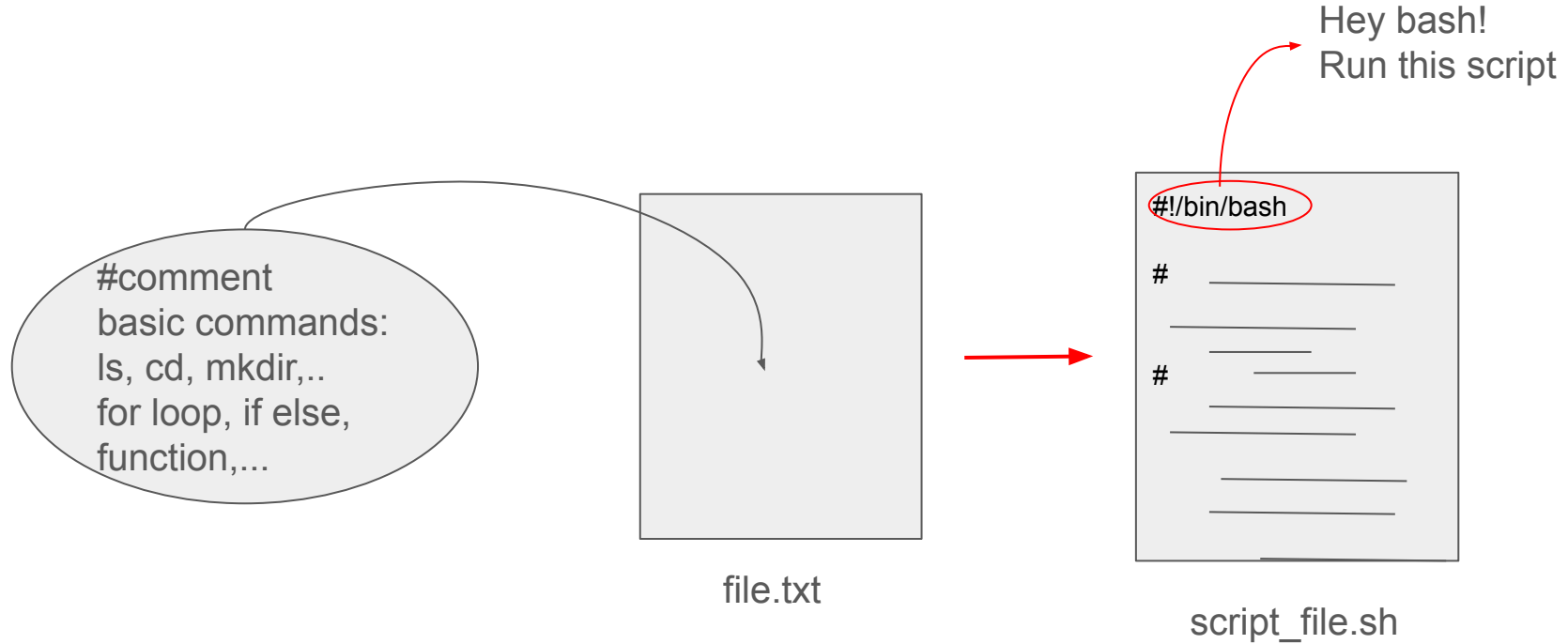
```
cat << 'EOF' > sequences.fasta
>seq1_gene001 organism=Kitasatospora
ATGCGTAGCTAGCTAGCTAGCTAG
CTAGCTAGCTAGCTAGCTAGCTAG
>seq2_gene002 organism=Levilactobacillus
TTACGACTAGCTAGCTAGCTAGCT
GCTAGCTAGCTAGCTAGCTAGCTA
>seq3_gene003
ATGGGGCTAGCTAGCTAGCTAGCT
EOF
```

1. **Redirection (>):** Find all features from the "Prodigal" program in `genes.gff` and save them to a new file named `prodigal_only.gff`.
2. **Appending (>>):** First, find all features on "scaffold_1" and save them to a file named `combined_scaffolds.gff`. *Then*, find all features on "scaffold_2" and *add* them to the `combined_scaffolds.gff` file. (Check the result with `cat combined_scaffolds.gff`).
3. **Pipeline (`cut | sort | uniq`):** Get a sorted, unique list of all the *program names* (e.g., Prodigal, AUGUSTUS) in the `genes.gff` file. (Hint: This is the 2nd column).
4. **Pipeline (`cut | sort | uniq | wc -l`):** *Count* how many unique program names are in `genes.gff`.
5. **Challenge Pipeline (`grep | cut`):** From `sequences.fasta`, create a clean list of just the sequence IDs (e.g., `seq1_gene001`, `seq2_gene002`, etc.) and save this list to a file called `seq_ids.txt`.
 - o (Hint 1: `grep` for the headers).
 - o (Hint 2: Pipe to `cut`. You'll need to tell `cut` to use a "space" as the delimiter (`-d ' '`) and get the first field (`-f 1`).
 - o (Hint 3: The output will still have the `>`. Can you figure out how to remove it? You could pipe it to `cut` *again*, this time using `-c 2-` which means "characters 2 through the end").

Intro: Why Bash for Bioinformatics?

1. **Big Data:** Bioinformatics data (FASTQ, BAM, VCF) is often too large to open in Excel or a text editor. The command line is built to handle huge files.
2. **Automation:** Do you need to run the same analysis on 100 samples? You can write a 5-line script to do it for you, error-free.
3. **Reproducibility:** A script is a perfect record of every step you took, which is essential for good science.
4. **HPC/Cloud:** Most bioinformatics tools and supercomputers (High-Performance Computing clusters) *require* you to use the command line.

What is bash script?



-Robust and reproducible

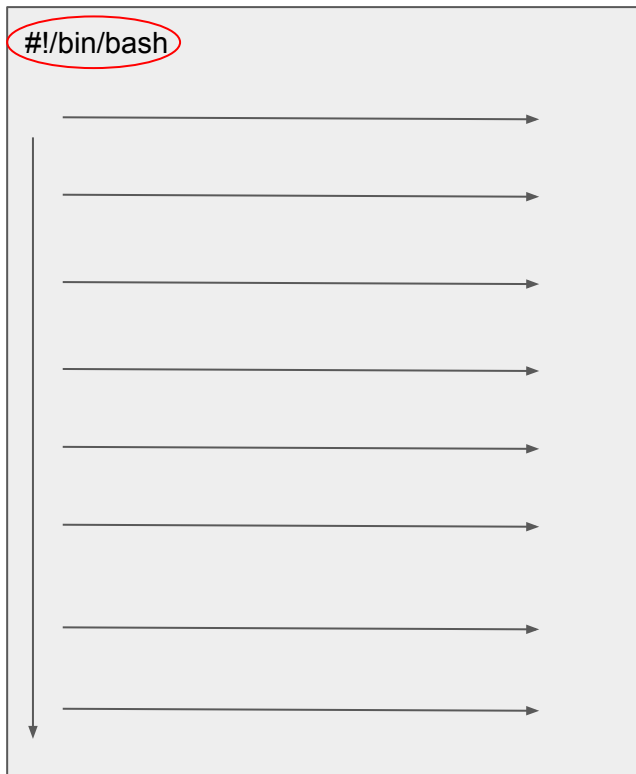
Source: Khainguyen

Why we need to document the command?

```
awk '
$1=="AC"{print"\n";sub(";",",",$0);print $2};
$1=="DE"{ORS=" ";print substr($0,6)}
/product/ {print"\n";ORS="\n";gsub("\\"",",",$0);$1="";print
substr($0,11)}
/translation/{gsub("/translation=",",",$0);gsub("\\"",",",$0);
while(i=1){$1="";print $0;getline; if ($1=="XX"){break} } }
/SQ/{getline;while(i=1){$NF="";print $0;getline; if
($1=="//"){break} } }
' U31362.1.gb
```

```
U31362
Human immunodeficiency virus type 1 isolate IND7 envelope glycoprotein gp120 (env) mRNA, partial cds.
envelope glycoprotein gp120
MGVRGILRNYQQWIIWIGILGFWMIMICNVVGNLWVTVYYGVVPWE
EAKTTLFCASDAKAYETEVEVHNWATHACVPTDPNPQEIFLENVTENFNMNRKNDMVNQMH
EDVISLWDQSLKPCVKLTPLCVTLCEQOVNVTSNGTOVNATNGEEIKNIKNSFNSTT
EIRDQRQTAYRLFYRLDLVPLDNKNGNSSSXYILINCNTSAITQACPKVTFDPIPIHYC
TPAGYAILKCNDKTFNGTGPCHNVSTVQCTHGKIPVSTQLLNGLAEEEEIIIRSEN
LDNVKTIIVHLNQSVEIVCTRPNNNTRKSIRIGPGQTFYATGDIIGDIIQAHNCISEAK
WNETLQRVRKKLAHFNPKNKTINFSSGGGLEITTHSFNCRGEFFYCNQSGLFNGTYMH
NGTKGNSSSVITIPCRKQIINMVGWGRAMYAPPIEGNITCKSNITGLLLVRDGGGLGP
SNDTETETFRPGGGMDRDNWRSELYKYKVVKIKPLGIAPTAKRRVVERE"
atggggagtg ggggggatact gaggaattat caacaatggt ggatatgggg catcttaggc
tttggatgt taatgatttg taatgtgta ggaacttgt gggtcacagt ctattatggg
gtacctgtgt gggagaagc aaaaactact ctattctgtg catcagatgc taagcatat
gagacagaag tgcataatgt ctgggctaca catgcctgtg taccacaga ccccaacca
caagaaatat tttgaaaa tgtaacagaa aattttaaca tgaggaaaaa tgacatggtg
aatcagatgc atgaggatgt aatcagttta tgggatcaaa gcctaaagcc ctgtgtaag
ttgacccac ctctgtgtac tttagaatgt agacaggtta atgttaccag taacggtaca
cagggttaacg ctaccagtaa tggagaagaa ataaaaata taaaaattg ctcttcaat
tcaaccacag aaataagaga taggaagcag acagcgtatc gacttttta tgcattgtat
ctagtaccac ttgataacaa gaatgggagc aactctagta agtatatatt aataaattgt
aataacctcag ccataacaca agcctgtcca aaggtcactt ttgatccaat tcctattcat
tattgtactc cagctgtgta tgcgattcta aagtgtaagt ataagacatt caatgggaca
ggaccatgcc ataattgtag cacagtacaa tgtacacatg gaattaagcc agtagtatca
actcaactac tgttaaatgg tagcctagca gaagaagaga taataattag atctgaaaaa
ctgacagaca atgtcaaaac aataatagta catcttaac aatctgtaga aattgtgtgt
acaagaccca acataataac aagaaaaagt ataaggatag gaccaggaca acaattctat
gcaacaggag acataatagg ggacataaga caagcacatt gtaacattag tgaagctaaa
tggaatgaaa ctttacaagg ggtaagaaaa aaattagcag aacacttccc taataaaaca
ataaatttta catcatctc agggaggggac ctagaattaa caacacatag ctttaattgt
agaggagaaat ttttctattg taatcaatca ggcctgttta atggtacata catgataat
ggtaacaaaag gtaattcaag ctcagtcac acaatcccat gcagaataaa gcaaatata
aacatgtggc agggggtagg acgagcaatg tatgccctc ccaattgaag aaacataaca
tgtaaatcaa atatcacagg actactattg gtacgtgatg gaggactagg accatcaaat
gacacagaga cagagacatt cagacctgga ggaggagata tgagggacaa ttggagaagt
gaattatata aatataaagt ggtaaaaatt aagccattgg gaatagcacc cactacagca
aaaaggagag tgggtggagag agaa
```

What is bash script?



script_file.sh

#!/bin/bash

#! called “shebang”

/bin/bash is the path to the bash program file

->It make bash to do the script command

If the commands in this are in python, it is called python script, R is called R script

Commands read from **left to right** and from **top to bottom**

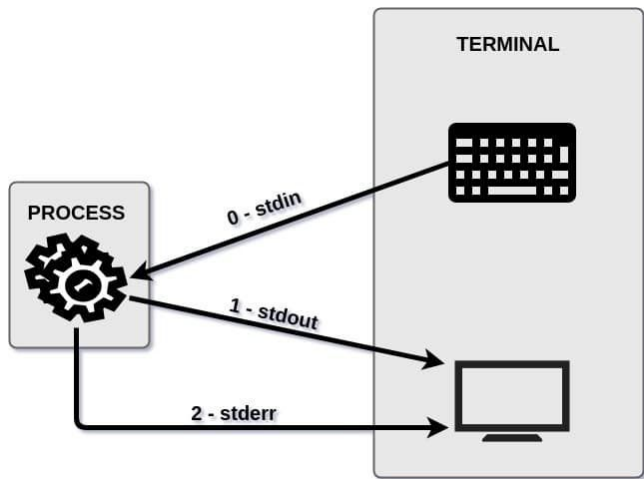
Write your first bash script

```
• (base) huyha@dummycomputer:~/huyha/homework/Huyha-Learning/baschscript$ nano myscript.sh
• (base) huyha@dummycomputer:~/huyha/homework/Huyha-Learning/baschscript$ ls -lh
total 4.0K
-rw-rw-r-- 1 huyha huyha 55 May  8 16:11 myscript.sh
• (base) huyha@dummycomputer:~/huyha/homework/Huyha-Learning/baschscript$ chmod u+wx myscript.sh
• (base) huyha@dummycomputer:~/huyha/homework/Huyha-Learning/baschscript$ ls -lh
total 4.0K
-rwxrw-r-- 1 huyha huyha 55 May  8 16:11 myscript.sh
• (base) huyha@dummycomputer:~/huyha/homework/Huyha-Learning/baschscript$ ./myscript.sh
Hello world
My first script
```

```
baschscript > $ myscript.sh
```

```
1  #!/bin/bash
2
3  echo "Hello world"
4  echo "My first script"
5
```

DATA STREAM



Run Linux command

- standard **input (stdin)**
- standard **output (stdout)** – displays the output
- standard **error (stderr)** – displays error output

```
(base) huyha@dummyscomputer:~/huyha/homework/Huyha-Learning/baschscript$ expr 1 + 1
2
ⓧ (base) huyha@dummyscomputer:~/huyha/homework/Huyha-Learning/baschscript$ cd ?
bash: cd: ?: No such file or directory
```

DATA STREAM

Operator	Name	What It Does	Common Use Case
>	Redirect stdout	<code>command > file.txt</code> (Same as <code>command 1> file.txt</code>)	Save the <i>results</i> of a command.
2>	Redirect stderr	<code>command 2> errors.txt</code>	Save <i>only the errors</i> of a command.
&>	Redirect All	<code>command &> all.log</code>	Save <i>everything</i> to one log file.
>>	Append stdout	<code>command >> file.txt</code>	Add results to the end of a file.
2>>	Append stderr	<code>command 2>> errors.txt</code>	Add errors to the end of a file.
&>>	Append All	<code>command &>> all.log</code>	Add everything to the end of a log.

Command	What It Does
<code>command > /dev/null</code>	Hides the normal output.
<code>command 2> /dev/null</code>	Hides <i>only</i> the error messages.
<code>command &> /dev/null</code>	Hides <i>everything</i> (all output and all errors).

Function syntax

1. The most widely used format is:

```
<function name> () {  
    <commands>  
}
```

Alternatively, the same function can be one line:

```
<function name> () { <commands>; }
```

2. The alternative way to write a bash function is using the reserved word **function**:

```
function <function name> {  
    <commands>  
}
```

Or in one line:

```
function <function name> { <commands>; }
```

Function example

```
baschscript > $ myscript.sh
```

```
1  #!/bin/bash
```

```
2  
3  function home () {  
4  cd /home/huyha  
5  }
```

```
6  
7  function home () { cd /home/huyha; }  
8
```

```
9  haha () {  
10 cd /home/huyha  
11 }
```

- (base) huyha@dummycomputer:~\$ cd \$a
- (base) huyha@dummycomputer:~/huyha/homework/Huyha-Learning/baschscript\$ home
- (base) huyha@dummycomputer:~\$ cd \$a
- (base) huyha@dummycomputer:~/huyha/homework/Huyha-Learning/baschscript\$ haha
- (base) huyha@dummycomputer:~\$

Variables, argument

Variables

\$name = variable



```
• (base) huyha@dummycomputer:~/huyha/homework/Huyha-Learning/baschscript$ echo "good morning huyha"
good morning huyha
• (base) huyha@dummycomputer:~/huyha/homework/Huyha-Learning/baschscript$ name=huyha
• (base) huyha@dummycomputer:~/huyha/homework/Huyha-Learning/baschscript$ echo "good morning name"
good morning name
• (base) huyha@dummycomputer:~/huyha/homework/Huyha-Learning/baschscript$ echo "good morning $huyha"
good morning huyha
• (base) huyha@dummycomputer:~/huyha/homework/Huyha-Learning/baschscript$ echo "good morning $name"
good morning huyha
```


Arguments

```
baschscript > $ myscript.sh
1  #!/bin/bash
2
3  #$0
4  echo "This script name is $0"
5
6  #
7  echo "The first argument: $1"
8  echo "The second argument: $2"
9  echo "The third argument: $3"
10 #
11 echo "Total argument: $#"
```

```
12 #
13 a=$1
14 b=$2
15 c=$3
```

```
16 echo "sum $(( $a + $b + $c ))"
```

```
• (base) huyha@dummyscomputer:~/huyha/homework/Huyha-Learning/baschscript$ ./myscript.sh 1 2 3
```

```
This script name is ./myscript.sh
```

```
The first argument: 1
```

```
The second argument: 2
```

```
The third argument: 3
```

```
Total argument: 3
```

```
6
```

Basic math

```
1  #!/bin/bash
2
3  expr 1 + 1
4
5  echo "1 + 1" | bc
6
7  echo $((1+1))
8  |
```

```
1  #!/bin/bash
2
3  # Addition
4  result=$((5 + 3))
5  echo "5 + 3 = $result"
6
7  # Subtraction
8  result=$((10 - 4))
9  echo "10 - 4 = $result"
10
11 # Multiplication
12 result=$((6 * 2))
13 echo "6 * 2 = $result"
14
15 # Division
16 result=$((20 / 5))
17 echo "20 / 5 = $result"
18
19 # Modulus
20 result=$((15 % 4))
21 echo "15 % 4 = $result"
22
```

Guess question 1

Script

```
1  #!/bin/bash
2
3  num1=5
4  num2=10
5  num3=$(( $num1+$num2 ))
6
7  echo $(( $num1 + $num2 - $num3 ))
8  echo "$num3"
9  echo " $1 % $num3" | bc
10
```

Input 1 `./question1.sh 10`

2 `./question1.sh 15`

Guess question 1

Script

```
1  #!/bin/bash
2
3  num1=5
4  num2=10
5  num3=$(( $num1+$num2 ))
6
7  echo $(( $num1 + $num2 - $num3 ))
8  echo "$num3"
9  echo " $1 % $num3" | bc
10
```

Input 1 `./question1.sh 10`

2 `./question1.sh 15`

TRUE OR **FALSE**

1. 0, 15, 10

2. 0, 15, 0

Result question 1

```
(base) huyha@dummycomputer:~/huyha/homework/Huyha-Learning/baschscript$ ./question1.sh 10
0
15
10
(base) huyha@dummycomputer:~/huyha/homework/Huyha-Learning/baschscript$ ./question1.sh 15
0
15
0
```

TRUE OR **FALSE**

1. 0, 15, 10
2. 0, 15, 0

Homework

1/ Use this code to down

```
mkdir ./homework
```

```
curl -o ./homework/fa1.txt https://drive.usercontent.google.com/download?id=1RvsIx3\_qGI\_znCAWHDyCHKfhUQxm5U6j
```

```
curl -o ./homework/fa2.txt https://drive.usercontent.google.com/download?id=1znAljOXBLeq4qt0asce185mzKkPgmxxv
```

or you can down 2 files by click to the drive link above

(If you don't have curl please install it.-(sudo apt install curl))

The files after down will be in the homework Directory use (cd ./homework) or (ls ./homework) to see it

Gulde

+ Use “Grep” to extract the organism name of fa1.txt and fa2.txt

+ Use “Grep” and “wc” to find the nucleotide length of file

+ Compare the nucleotide length between fa1.txt and fa2.txt

2/ Output the **organism name** and **length** of the bigger nucleotide length file

```
1 >KT719404.1 Porcine circovirus 2 strain PCV2, complete genome
2 ACCAGCGCACTTCGGCAGCGGCAGCACCTCGGCAGCACCTCAGCAGCAACATGCCAGCAAGAAGATGG
3 AAGAAGCGGACCCCAACCCATAAAAGGTGGGTGTTCACTTTGAATAATCCTCCGAAGACGAGCGCAAG
4 AAAATACGGGAGCTTCCAATCTCCCTGTTTGATTATTTTATTGTTGGCGAGGAGGGTAATGAGGAAGGAC
5 GAACACCTCACCTCCAGGGTTCGCTAATTTTGTGAAAAAGCAAACTTTAAATAAAGTGAAGTGGTATCT
6 TGGTGCCCGCTGCCACATCGAGAAAGCCAAAGGAAGTATCAGCAGAATAAAGAATATTGCAGTAAAGAA
7 GGCAACTTACTTATTGAGTGTGGAGCTCCTAGATCTCAAGGACAACGGAGTGACCTGTCTACTGCTGTGA
8 GTACCTTGTGGAGAGCGGGAGTCTGGTGACCGTTGCAGAGCAGCACCCCTGTAACGTTTGTGAGAAATTT
9 CCGCGGGCTGGCTGAACTTTGTAAAGTGAGCGGGAAAATGCAGAAGCGTGATTGGAAGACTAATGTACAC
10 GTCATTGTGGGGCCACCTGGGTGTGGTAAAAGCAAATGGGCTGCTAATTTTGCAGA
11
```